SIMATS ENGINEERING

Descriptive Written Exam

COURSE NAME: Software Testing **COURSE CODE: CSA37**

COURSE OUTCOME COVERED

CO1: *Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.*

Assessment Tool Weightage: 50%

Sl. No	Questions	Bloom's Level
1	A banking application team is facing frequent failures due to unstructured testing practices. As a test engineer, apply the concept of testing as an engineering activity to redesign the testing approach. Explain how systematic test design, tools, and standards can be introduced. Illustrate how this approach improves defect detection and quality. Suggest suitable techniques to replace ad-hoc testing. Justify your solution with a practical example.	3
2	An ATM system is producing inconsistent results due to lack of a defined testing process. As a tester, demonstrate how you would implement a structured testing process across SDLC phases. Explain each stage and its relevance to quality improvement. Apply your approach to ATM withdrawal functionality. Discuss how process consistency reduces defects. Conclude with expected outcomes.	3
3	A web application shows mismatch between expected and actual outputs during login and transaction operations. Apply basic testing definitions to identify errors, defects, and failures in this scenario. Illustrate their relationships with suitable examples. Explain how improper understanding of these terms affects testing. Suggest how to improve defect reporting clarity. Justify your answer with examples.	3
4	A software project team observes that most defects are concentrated in a few modules, leading to delays. Apply software testing principles to address this issue. Explain how defect clustering and other principles guide test planning. Illustrate how test cases can be prioritized effectively. Discuss how applying these principles improves testing efficiency. Provide a real-time example.	3

5	A company maintains a defect repository but fails to utilize it effectively, resulting in repeated defects across releases. As a tester, apply defect classification and repository concepts to improve the situation. Explain how defects originate and how they can be categorized. Illustrate how repository data can support test design and prevention. Suggest improvements to avoid recurring defects. Conclude with expected quality improvements.	3
6	A mobile banking application frequently crashes during fund transfer operations. As a tester, apply software testing principles to identify the root cause of failures. Explain how systematic testing can reduce such issues. Illustrate your approach with suitable examples. Suggest improvements in testing practices to avoid future failures.	3
7	A development team confuses debugging with testing, leading to ineffective defect resolution. Apply the concepts of testing and debugging to differentiate both processes. Explain their roles in software quality. Illustrate how improper usage affects defect detection. Suggest a structured approach to integrate both effectively.	3
8	An e-commerce platform shows inconsistent behavior across browsers. Apply testing principles to design a strategy for cross-browser testing. Explain how defects originate in such scenarios. Illustrate how systematic testing improves reliability. Suggest tools and techniques for better coverage.	3
9	A payroll system produces incorrect salary calculations due to logical errors in code. Apply defect classification techniques to categorize these defects. Explain how understanding defect classes improves testing. Illustrate how such defects can be prevented in future releases. Suggest improvements in testing strategy.	3
10	A software team skips proper test case design and directly executes random tests. Apply testing as an engineering activity to redesign the testing process. Explain the importance of structured test case design. Illustrate how this improves defect detection. Suggest suitable techniques to enhance test effectiveness.	3

11	A healthcare application shows data inconsistency across modules. Apply testing principles to analyze the issue. Explain how defects may originate during development. Illustrate how systematic testing ensures data integrity. Suggest improvements in defect tracking and reporting.	3
12	A company faces repeated defects in its software releases despite testing efforts. Apply the concept of defect repository to analyze the issue. Explain how repository data supports test design. Illustrate how defect patterns can be identified. Suggest improvements to prevent recurring defects.	3
13	A web-based learning system fails during peak usage due to untested scenarios. Apply software testing principles to identify gaps in testing. Explain how test planning and coverage impact performance. Illustrate how better test strategies improve system reliability. Suggest enhancements to avoid such failures.	3
14	A team observes that testers are not involved in early development stages. Apply the role of tester in SDLC to improve this situation. Explain how early involvement impacts quality. Illustrate benefits using a practical example. Suggest changes in team collaboration practices.	3
15	A system produces correct outputs in some cases but fails in others due to incomplete test coverage. Apply testing principles to identify gaps. Explain how proper coverage ensures reliability. Illustrate how to improve test case design. Suggest techniques for better validation.	3
16	A logistics application shows delay in defect identification due to poor reporting practices. Apply testing concepts to improve defect reporting. Explain how clear definitions of defects help in tracking. Illustrate how structured reports enhance communication. Suggest improvements in reporting standards.	3
17	A software team struggles to maintain consistency in testing across modules. Apply process-based testing approaches to solve the issue. Explain how standardized processes improve quality. Illustrate your approach with an example. Suggest methods to ensure consistency.	3

18	A system fails after deployment due to untested edge cases. Apply testing principles to identify why such cases were missed. Explain how boundary testing helps in such scenarios. Illustrate how to include edge cases in testing. Suggest improvements in test design.	3
19	A company lacks proper documentation of defects, making it difficult to track issues. Apply defect repository concepts to address this problem. Explain how documentation improves defect management. Illustrate how structured repositories support analysis. Suggest improvements in documentation practices.	3
20	A banking system shows repeated failures in transaction processing due to improper testing practices. Apply software testing principles to redesign the testing approach. Explain how structured testing improves quality. Illustrate your solution with examples. Suggest techniques to enhance defect detection.	3

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

/

SOFTWARE TESTING. COIMBATORE Name: Logha

13/10/26 SUB. CODE: CSA 3714 Reg. No: 192312130.

4, 9, 11, 15, 18

(4.) Software testing principles helps testers use their time and resources efficiently. One important principle is defect clustering. which means large proportions of defects are often concentrated in a small no. of modules.

What to do?

The tester should analyze historical defect reports to identify modules with frequent failures and these modules should receive detailed testing.

eg: If the fund transfer module contains most banking defects. prioritize transaction validation and network failure scenarios etc.

Other principles.

* Dated defects during requirements and design reviews.
* Select representative and high-risk test cases.

Prioritize test cases: test cases can be ranked according to business impact and defect history.

---

9. Defect classification: Process of grouping software defects according to their nature, cause or effect. In a payroll system, incorrect salary calculations are primarily logical and functional defects.

1. Classify the defect.
2. Identify the cause:

the defect may originate from misunderstood payroll requirements, incorrect formulas

3. Design appropriate test cases.

test cases should cover.

* Basic salary and allowance.
* Tax & other deduction.
* Min and Max salary values.
* Decimal calculations.

4. Prevent future defects. use requirement reviews, unit testing of salary formulas, code reviews.

eg: If basic salary is ₹30,000, allowances are ₹5000 and deductions are ₹3000, the expected net salary is ₹32,000. If the system adds deductions instead, it gives ₹38,000

---

11. Data integrity means maintaining the accuracy and ~~the~~ to avoid the data inconsistency throughout the application. In healthcare department incorrect or inconsistant data can affect the appointments, prescription and medical records.

Identify the possible defects.

* Incorrect requirement or database design.
* Failed database updates.
* Synchronization failures.

Apply systematic testing:

The department should have a standard testing methods & possible defects that can occurs.

And to verify if the data remains the same after creating updating and retrieving patient records.

Verify data consistency:

Check whether the data remains unchanged or changed during updating or after prescriptions. as it can seriously affect the system.

Improve defect tracking:

Maintain a defect repository containing the affected modules, patient-data test case, expected and actual outcome, reproduction steps and evidence.

---

15. Test coverage measures how much of the specified software. functionality, requirements or other test objectives has been exercised by testing. Incomplete test coverage means some important scenarios has been missed.

Identify coverage gaps.

Failure may occurs because testers only check basic input, ~~Invali~~ ignore invalid test cases or less frequently used functions.

Improve test case design.

Map test cases according to the requirements.

Apply boundary value analysis to test the limits.

Include negative and exception scenarios.

Prioritize high-risk functions.

Measure & Review coverage:

Track which requirements & scenarios have been tested and which remains uncovered.

Improved coverage exposes more defects, validates more requirements and reduces the unexpected failures after deployment.

* Proper test case design and coverage analysis improve defect detection and increase software reliability.

---

18. An edge case is a situation at an extreme, unusual or exceptional condition that may expose defects. ~~These~~.

Software fails after deployment because of unusual or extreme inputs were not tested.

Boundry value analysis focusses on values at and around the given limit of an i/p range.

Testers may focus on normal, basic i/p, use insufficient test data or fail to concides min and max limits, empty value, large i/p and exception conditions.

Boundry value analysis:

Suppose if the system's range is ₹100 to ₹10,000 check at ₹99 & ₹11,000 & ₹100 & ₹10,000 to see if the i/p o/p are correct and the expected outcome has seen.

* Check empty values and around the limits to see how it performs.